

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH & TRUYỀN THÔNG



BÁO CÁO ĐỒ ÁN CHUYÊN NGÀNH

**Phát triển hệ thống quản lý tài liệu bảo mật
dựa trên kiến trúc Microservices và quy trình
DevSecOps**

Môn học NT114 - Đồ án chuyên ngành
Giảng viên hướng dẫn: ThS. Lê Anh Tuấn

Thành viên	MSSV
Huỳnh Lê Đại Thắng	23521422
Nguyễn Trường Duy	23520380

TP. Hồ Chí Minh, tháng 06 năm 2026

Lời cảm ơn

Trong quá trình thực hiện đồ án chuyên ngành với đề tài “Phát triển hệ thống quản lý tài liệu bảo mật dựa trên kiến trúc Microservices và quy trình DevSecOps”, nhóm đã có dịp đưa kiến thức về phát triển phần mềm, kiến trúc microservices, bảo mật ứng dụng và tự động hóa triển khai vào một hệ thống tương đối hoàn chỉnh. Quá trình này có không ít khó khăn, từ giới hạn tài nguyên và chi phí cho đến các vấn đề phát sinh khi triển khai và vận hành hệ thống. Vượt qua những trở ngại đó là một phần đáng giá của trải nghiệm, giúp nhóm trưởng thành hơn trong cách xây dựng và vận hành một hệ thống thực tế.

Nhóm xin gửi lời cảm ơn đến thầy ThS. Lê Anh Tuấn đã định hướng, góp ý và hỗ trợ nhóm trong quá trình xác định phạm vi, thiết kế kiến trúc và hoàn thiện sản phẩm. Những góp ý trong quá trình thực hiện giúp nhóm nhìn nhận rõ hơn mối liên hệ giữa yêu cầu nghiệp vụ, yêu cầu bảo mật và khả năng vận hành của hệ thống.

Nhóm cũng xin cảm ơn nhà trường và khoa đã tạo điều kiện để nhóm được thực hiện đồ án theo hướng thực hành, qua đó rèn luyện kỹ năng xây dựng hệ thống, kiểm thử, triển khai và trình bày kết quả kỹ thuật một cách có hệ thống. Mặc dù đã cố gắng hoàn thiện, báo cáo và sản phẩm vẫn có thể còn thiếu sót. Nhóm mong nhận được góp ý để tiếp tục cải thiện hệ thống trong các giai đoạn sau.

Tóm tắt

DocVault là một hệ thống quản lý tài liệu bảo mật được xây dựng theo kiến trúc microservices, hướng đến bài toán quản lý vòng đời tài liệu trong môi trường có yêu cầu phân quyền, kiểm soát tải xuống, lưu vết audit và hỗ trợ compliance. Hệ thống cho phép người dùng tạo tài liệu, tải lên phiên bản file, gửi duyệt, phê duyệt, xuất bản, lưu trữ và truy vết các hoạt động liên quan. Các vai trò chính gồm Viewer, Editor, Approver, Compliance Officer và Admin. Điểm nhóm muốn nhấn mạnh là quyền truy cập không được quyết định bởi một yếu tố đơn lẻ: nó là kết quả tổng hợp của RBAC, ACL, trạng thái tài liệu và mức phân loại bảo mật, và được thực thi ở backend.

Về mặt ứng dụng, DocVault gồm một frontend Next.js, một API Gateway và năm microservice NestJS lần lượt lo metadata, nội dung file, workflow, audit và notification. Xác thực dùng Keycloak; metadata nằm ở PostgreSQL, audit log ở MongoDB, còn file ở MinIO/S3. Trong các kiểm soát bảo mật, nhóm dành nhiều công nhất cho ba thứ: ranh giới Compliance Officer metadata-only, grant token ngắn hạn cho preview/download, và audit hash-chain để chuỗi sự kiện kiểm tra được tính toàn vẹn; các phần như ACL deny-overrides, malware scanning, DLP và retention evidence bổ trợ quanh ba trục đó.

Về mặt DevSecOps, nhóm dựng một pipeline Jenkins chạy từ kiểm thử và các bước quét bảo mật (SAST, SCA, quét filesystem/image, IaC, validate Terraform) cho tới build image, cập nhật GitOps, kiểm tra Argo CD và quét động bằng OWASP ZAP sau triển khai. Cả hai hợp phần đều đã chạy được và có bằng chứng vận hành thật; phần còn lại chủ yếu là gia cố cho quy mô production, chẳng hạn chuẩn hóa provenance, bắt buộc xác minh chữ ký tại admission và diễn tập khôi phục dữ liệu.

Từ khóa: DocVault, quản lý tài liệu, bảo mật ứng dụng, microservices, DevSecOps, RBAC, ACL, audit, DLP, Jenkins, GitOps.

Abstract

DocVault is a secure document management system designed around a microservices architecture and a DevSecOps-oriented delivery process. The system addresses document lifecycle management in an environment that requires role-based access control, controlled file access, auditability, and compliance evidence. Core workflows include document creation, file upload and versioning, review submission, approval, publication, archiving, preview/download authorization, audit inspection, and evidence export. A central design point is that an access decision is never made from a single attribute: it is the combined result of role, ACL, document status, and classification, and it is always enforced at the backend rather than in the user interface.

The application layer consists of a Next.js frontend, an API Gateway, and five NestJS microservices handling metadata, file content, workflow, audit, and notification. Keycloak provides identity and access management; metadata lives in PostgreSQL, audit events in MongoDB, and file objects in MinIO/S3. Among the security controls, three received the most attention: the metadata-only boundary for the Compliance Officer, short-lived grant tokens for preview/download, and a tamper-evident audit hash chain; ACL deny-overrides, malware scanning, DLP, and retention evidence support these three.

On the DevSecOps side, a Jenkins pipeline runs from testing and security scans (SAST, SCA, filesystem and image scanning, IaC, Terraform validation) through image build, GitOps updates, Argo CD health checks, and post-deploy dynamic scanning with OWASP ZAP. Both scopes are implemented and backed by real operational evidence; the remaining work is mostly production-grade hardening, such as standardized provenance, admission-time signature enforcement, and data recovery drills.

Keywords: DocVault, document management, application security, microservices, DevSecOps, RBAC, ACL, audit, DLP, Jenkins, GitOps.

Mục lục

Danh sách hình vẽ

Danh sách bảng

Danh mục từ viết tắt

Từ viết tắt	Tiếng Anh	Diễn giải trong báo cáo
DMS	Document Management System	Hệ thống quản lý tài liệu.
RBAC	Role-Based Access Control	Kiểm soát truy cập theo vai trò.
ACL	Access Control List	Danh sách quyền truy cập theo người dùng, vai trò, nhóm hoặc tất cả.
OIDC	OpenID Connect	Lớp định danh dùng trên OAuth 2.0, được Keycloak sử dụng cho đăng nhập.
JWT	JSON Web Token	Token mang thông tin định danh và vai trò của người dùng.
CI/CD	Continuous Integration / Continuous Delivery	Tích hợp, kiểm thử, đóng gói và triển khai tự động.
DevSecOps	Development, Security, Operations	Tích hợp bảo mật vào quy trình phát triển và vận hành.
SAST	Static Application Security Testing	Kiểm thử bảo mật bằng phân tích mã nguồn tĩnh.
SCA	Software Composition Analysis	Phân tích thành phần phụ thuộc và lỗ hổng thư viện.
DAST	Dynamic Application Security Testing	Kiểm thử bảo mật động trên ứng dụng đang chạy.
IaC	Infrastructure as Code	Mô tả hạ tầng bằng mã, ví dụ Terraform/Kubernetes manifest.
SBOM	Software Bill of Materials	Danh mục thành phần phần mềm và phụ thuộc.
DLP	Data Loss Prevention	Cơ chế phát hiện và giảm rủi ro rò rỉ dữ liệu nhạy cảm.
EKS	Elastic Kubernetes Service	Dịch vụ Kubernetes được quản lý trên AWS.

GitOps	Git Operations	Mô hình dùng Git làm nguồn sự thật cho trạng thái triển khai.
API	Application Programming Interface	Giao diện lập trình ứng dụng giữa frontend, gateway và các service.
AWS	Amazon Web Services	Nền tảng điện toán đám mây dùng để triển khai hạ tầng đề tài.
ERD	Entity Relationship Diagram	Sơ đồ quan hệ thực thể mô tả dữ liệu và ranh giới sở hữu.
VPC	Virtual Private Cloud	Mạng riêng ảo trên AWS chứa các subnet của cụm EKS.
NAT	Network Address Translation	Cổng cho phép node trong private subnet truy cập Internet ra ngoài.
CIDR	Classless Inter-Domain Routing	Cách biểu diễn dải địa chỉ IP, dùng giới hạn truy cập Kubernetes API.
TLS	Transport Layer Security	Giao thức mã hóa kênh truyền cho lưu lượng HTTPS.
S3	Simple Storage Service	Dịch vụ lưu trữ object của AWS dùng cho file tài liệu và backup.
KMS	Key Management Service	Dịch vụ quản lý khóa mã hóa của AWS.
SSE-KMS	Server-Side Encryption with KMS	Mã hóa phía máy chủ cho object S3 bằng khóa KMS.
IRSA	IAM Roles for Service Accounts	Cấp quyền AWS cho từng Kubernetes ServiceAccount qua OIDC.
IAM	Identity and Access Management	Dịch vụ quản lý định danh và quyền truy cập của AWS.
IMDSv2	Instance Metadata Service version 2	Phiên bản metadata service yêu cầu token, giảm rủi ro SSRF trên EC2.
CVE	Common Vulnerabilities and Exposures	Định danh chuẩn cho lỗ hổng bảo mật đã được công bố.
CVSS	Common Vulnerability Scoring System	Hệ thống chấm điểm mức độ nghiêm trọng của lỗ hổng.
SLSA	Supply-chain Levels for Software Artifacts	Khung chuẩn hóa provenance của artifact trong chuỗi cung ứng.
RPO	Recovery Point Objective	Mức dữ liệu tối đa chấp nhận mất khi khôi phục sau sự cố.
RTO	Recovery Time Objective	Thời gian tối đa chấp nhận để khôi phục dịch vụ sau sự cố.

MVP	Minimum Viable Product	Sản phẩm khả dụng tối thiểu đủ để minh họa nghiệp vụ cốt lõi.
-----	------------------------	---

Chương 1

Tổng quan đề tài

1.1 Bối cảnh

Tài liệu trong một tổ chức không chỉ là file để lưu và tìm lại. Nó có vòng đời: được tạo, chỉnh sửa, gửi duyệt, xuất bản rồi lưu trữ; và ở mỗi bước, việc ai được xem, ai được tải, ai được phê duyệt đều cần có ranh giới rõ ràng. Tài liệu cũng có thể là hợp đồng, báo cáo tài chính, hồ sơ nhân sự hay biên bản phê duyệt, tức là loại dữ liệu mà mọi thao tác lên nó nên để lại dấu vết để sau này còn truy lại được.

Các hệ thống quản lý tài liệu phổ biến đã làm tốt việc lưu trữ, phân quyền cơ bản và tìm kiếm. Nhóm chọn đề tài DocVault không nhằm dựng lại một hệ thống đầy đủ tính năng như vậy, mà muốn đi sâu hơn vào những khía cạnh nhóm thấy đáng học và thường ít được đặt làm trọng tâm: quản lý vòng đời tài liệu một cách có kiểm soát, kiểm soát truy cập nhiều lớp theo mức nhạy cảm của tài liệu, ghi nhận audit log có thể truy vết và kiểm tra tính toàn vẹn, và đưa toàn bộ quy trình phát hành vào một pipeline DevSecOps có bằng chứng. Nói cách khác, mục tiêu của nhóm thiên về chiều sâu bảo mật, khả năng truy vết và quy trình, hơn là bề rộng tính năng.

Một nguyên tắc nhóm đặt ra từ đầu là chính sách bảo mật phải được thực thi ở backend. Giao diện có thể ẩn hoặc làm mờ thao tác để gọn gàng hơn, nhưng quyết định cho phép hay từ chối phải nằm ở phía service và được kiểm thử tại đó, vì với tài liệu nhạy cảm thì một thao tác bị bỏ lọt ở backend mới là rủi ro thật, còn việc ẩn nút trên màn hình chỉ là cải thiện trải nghiệm.

Mặt thứ hai của đề tài là quy trình phát hành. Nhóm chọn DevSecOps không phải vì đó là từ khóa thời thượng, mà vì nếu kiểm soát bảo mật chỉ được kiểm tra một lần ở cuối thì rất khó lặp lại và dễ bỏ sót khi code thay đổi. DevSecOps đưa các bước kiểm tra vào sớm và rải đều khắp vòng đời: mã nguồn, thư viện phụ thuộc, cấu hình hạ tầng, Docker image, rồi đến ứng dụng đang chạy sau khi triển khai. NIST SSDF và hướng dẫn DevSecOps của OWASP cũng đi theo tư tưởng này: tự động hóa kiểm thử và thu thập bằng chứng để giảm rủi ro và làm cho mỗi lần phát hành có thể kiểm chứng lại được.

`nist_ssdf`, `owasp_devsecops`.

DocVault vì vậy được đặt ra với hai phần gắn vào nhau: một hệ thống quản lý tài liệu thực thi chính sách bảo mật ở backend, và một quy trình DevSecOps lo việc kiểm thử, đóng gói và triển khai hệ thống đó một cách có dấu vết.

1.2 Vấn đề cần giải quyết

Đề tài tách thành hai nhóm vấn đề. Nhóm ứng dụng là xây dựng một hệ thống quản lý tài liệu có phân quyền rõ, có quy trình phê duyệt, lưu file an toàn và có audit trail đủ để truy vết. Nhóm quy trình là đưa kiểm soát bảo mật vào pipeline, sao cho mỗi lần đổi mã nguồn đều được kiểm tra tự động thay vì phụ thuộc vào việc nhớ chạy thủ công. Hai nhóm này phải làm song song, vì một hệ thống bảo mật tốt mà phát hành thủ công thì vẫn dễ trượt ở khâu triển khai.

Cụ thể hơn, nhóm rút ra các yêu cầu sau khi phân tích những câu hỏi nêu ở phần bối cảnh:

- Người dùng thuộc các vai trò khác nhau phải nhìn thấy và thao tác trên tài liệu theo đúng phạm vi quyền hạn.
- Tài liệu hỗ trợ bốn mức phân loại: public, internal, confidential và secret.
- Luồng nghiệp vụ phải hỗ trợ tạo bản nháp, tải file, gửi duyệt, phê duyệt, từ chối và lưu trữ.
- File nhạy cảm không nên được cấp URL tải trực tiếp nếu cần đi qua đường stream có kiểm soát.
- Compliance Officer được xem metadata, audit và bằng chứng tuân thủ, nhưng không được truy cập nội dung file.
- Các thao tác quan trọng phải được audit và có khả năng kiểm tra tính toàn vẹn.
- Pipeline phải có các bước kiểm thử và quét bảo mật trước, trong và sau khi build/deploy.

Trong số này, ranh giới của Compliance Officer là yêu cầu nhóm cân nhắc nhiều nhất: một người có quyền đọc toàn bộ audit log rất dễ được mặc định cho luôn quyền đọc file, nhưng đó lại chính là lỗ hổng mà vai trò compliance cần tránh.

1.3 Mục tiêu đề tài

Mục tiêu tổng quát của đề tài là xây dựng và trình bày một hệ thống quản lý tài liệu bảo mật tên DocVault theo hướng microservices, có tích hợp các kiểm soát bảo mật ở cả runtime và quy trình DevSecOps.

Các mục tiêu cụ thể gồm:

1. Thiết kế kiến trúc tổng thể cho hệ thống quản lý tài liệu bảo mật, gồm frontend, API Gateway, các microservice nghiệp vụ và các thành phần hạ tầng.
2. Xây dựng các chức năng chính của hệ thống: đăng nhập, quản lý tài liệu, version file, workflow phê duyệt, phân quyền, preview/download, audit, security dashboard, evidence và retention.
3. Thiết kế mô hình phân quyền dựa trên RBAC, ACL, trạng thái tài liệu, phân loại bảo mật và quy tắc deny-overrides.
4. Xây dựng cơ chế audit hash-chain và các bằng chứng phục vụ compliance.
5. Tích hợp các kiểm soát bảo mật như malware scanning, DLP, grant token ngắn hạn và kiểm soát truy cập nội dung file.
6. Xây dựng pipeline DevSecOps với Jenkins, SonarQube, Dependency Check, Trivy, Checkov, Terraform validation, Docker build/scan, GitOps, Argo CD, smoke test và OWASP ZAP.
7. Kiểm thử và đánh giá hệ thống dựa trên các kịch bản runtime, security evidence và pipeline evidence.

1.4 Phạm vi thực hiện

Phạm vi đề tài gồm hai hợp phần có quan hệ chặt chẽ. Hợp phần ứng dụng bao gồm giao diện web, các dịch vụ backend, luồng nghiệp vụ tài liệu, kiểm soát truy cập, audit, DLP, quét mã độc, retention và evidence center. Hợp phần DevSecOps bao gồm hạ tầng AWS EKS được quản lý bằng Terraform, quy trình CI/CD trên Jenkins, kho ảnh Harbor, GitOps với Argo CD, các cổng kiểm soát bảo mật và cơ chế giám sát sau triển khai.

Tại thời điểm hoàn thành báo cáo, pipeline đã tích hợp SAST, SCA, quét secret, Trivy, Checkov, Terraform Validate, Kyverno CLI, build và quét container image, đẩy artifact lên Harbor, cập nhật GitOps, kiểm tra trạng thái Argo CD, smoke test và DAST bằng OWASP ZAP. Hạ tầng triển khai đã sử dụng Amazon S3 với SSE-KMS, IAM Role for Service Account (IRSA), AWS Secrets Manager kết hợp External Secrets Operator, backup cơ sở dữ liệu, Prometheus, Grafana và Loki. Harbor đã lưu thông tin SBOM và trạng thái ký của artifact. Các nội dung còn lại tập trung vào gia cố vận hành ở quy mô production, thay vì bổ sung các thành phần nền tảng của quy trình.

Bảng 1.1: Phạm vi và trạng thái thực hiện của đề tài

Nhóm nội dung	Mô tả	Trạng thái
Ứng dụng web	Frontend Next.js, các màn hình dashboard, documents, approvals, audit, security, evidence, retention.	Đã triển khai
Backend microservices	Gateway, metadata-service, document-service, workflow-service, audit-service, notification-service.	Đã triển khai
Bảo mật runtime	RBAC, ACL, grant token, compliance boundary, DLP, malware scanning, audit hash-chain.	Đã triển khai
Local infrastructure	Postgres, MongoDB, MinIO, Keycloak qua Docker Compose.	Đã triển khai
DevSecOps pipeline	Jenkins pipeline với các stage kiểm thử, quét bảo mật, build, Harbor, GitOps, Argo CD, smoke test và DAST.	Đã triển khai
Hạ tầng cloud-native	Terraform, EKS, S3/KMS, IRSA, External Secrets, backup, Harbor và observability.	Đã triển khai
Gia cố vận hành production	Remote Terraform state, private node, admission enforcement, provenance chuẩn hóa, NetworkPolicy giới hạn egress, log persistence và diễn tập khôi phục.	Đang hoàn thiện

1.5 Đóng góp của đề tài

Đề tài có bốn đóng góp. Thứ nhất là một hệ thống quản lý tài liệu chạy được, bao phủ luồng từ khởi tạo, tải lên, phê duyệt đến lưu trữ. Thứ hai là mô hình kiểm soát truy cập nhiều lớp, kết hợp vai trò, ACL, mức phân loại, trạng thái và quyền sở hữu thay vì chỉ dựa vào một trong số đó. Thứ ba là cách đưa yêu cầu compliance vào ngay trong thiết kế: audit hash-chain để phát hiện sửa đổi, retention evidence và evidence packet metadata-only. Thứ tư, và cũng là phần nhóm đầu tư nhiều công nhất, là một quy trình DevSecOps truy vết được từ mã nguồn qua container artifact và GitOps revision đến workload trên EKS, với các cổng kiểm soát chất lượng, bảo mật chuỗi cung ứng, quản lý bí mật và giám sát vận hành gắn vào từng chặng.

1.6 Phân công công việc

Đề tài do hai thành viên cùng thực hiện theo hình thức đồng phụ trách: cả hai cùng tham gia khảo sát, thiết kế và kiểm thử toàn hệ thống, đồng thời mỗi người nhận vai trò chính ở một số hạng mục để bảo đảm tiến độ. Bảng 1.2 tóm tắt phân công theo vai trò chính và vai trò phối hợp.

Bảng 1.2: Phân công công việc giữa các thành viên

Hạng mục	Phụ trách chính	Phối hợp
Thiết kế kiến trúc tổng thể	Cả hai	—
Frontend Next.js và permission-aware UI	Huỳnh Lê Đại Thắng	Nguyễn Trường Duy
Backend microservices và bảo mật runtime	Huỳnh Lê Đại Thắng	Nguyễn Trường Duy
Mô hình phân quyền (RBAC, ACL, classification)	Huỳnh Lê Đại Thắng	Nguyễn Trường Duy
Audit hash-chain, DLP và evidence	Cả hai	—
Pipeline DevSecOps (Jenkins, các security gate)	Nguyễn Trường Duy	Huỳnh Lê Đại Thắng
Hạ tầng cloud-native (Terraform, EKS, GitOps)	Nguyễn Trường Duy	Huỳnh Lê Đại Thắng
Quản lý secret, registry và observability	Nguyễn Trường Duy	Huỳnh Lê Đại Thắng
Kiểm thử, thu thập bằng chứng và viết báo cáo	Cả hai	—

1.7 Cấu trúc báo cáo

Báo cáo gồm tám chương chính. Chương 1 trình bày bối cảnh, mục tiêu và phạm vi đề tài. Chương 2 trình bày cơ sở lý thuyết và công nghệ nền tảng. Chương 3 phân tích yêu cầu và thiết kế tổng thể. Chương 4 mô tả thiết kế dữ liệu, API và chính sách bảo mật. Chương 5 trình bày quá trình triển khai ứng dụng DocVault. Chương 6 trình bày quy trình DevSecOps và hạ tầng triển khai. Chương 7 mô tả kiểm thử, đánh giá và bằng chứng. Chương 8 tổng kết kết quả đạt được, hạn chế và hướng phát triển.

Chương 2

Cơ sở lý thuyết và công nghệ nền tảng

2.1 Hệ thống quản lý tài liệu bảo mật

Hệ thống quản lý tài liệu (Document Management System - DMS) lo việc lưu trữ, tổ chức, tìm kiếm, chia sẻ và kiểm soát vòng đời tài liệu; trong môi trường doanh nghiệp nó quản lý cả file lẫn metadata, phiên bản, người sở hữu, trạng thái phê duyệt và lịch sử thao tác. Phần lý thuyết này không nhắc lại định nghĩa đó dài dòng, mà tập trung vào điều khiến một DMS trở nên *bảo mật*.

Khác biệt nằm ở chỗ thực thi chính sách. Một tài liệu nhạy cảm không thể chỉ được bảo vệ bằng nơi lưu trữ; nó phải được bảo vệ bằng một chính sách truy cập nhất quán ở backend, đặc biệt với các thao tác như preview, stream, presign download, export evidence hay truy vấn audit. Rủi ro cần tránh là tình huống frontend chỉ ẩn nút thao tác trong khi backend vẫn nhận lời gọi API, vì khi đó chính sách bảo mật không còn đáng tin.

Vì lý do đó, DocVault đặt backend làm nơi ra quyết định cuối cùng. Frontend được phép hiển thị hoặc làm mờ thao tác để trải nghiệm tốt hơn, nhưng việc cho phép hay từ chối luôn thuộc về metadata-service, document-service, gateway và audit-service. Nguyên tắc này chi phối gần như mọi thiết kế ở các chương sau.

2.2 Khảo sát các hệ thống tương tự

Trên thị trường đã có nhiều hệ thống quản lý tài liệu, từ giải pháp thương mại đến mã nguồn mở. Việc đối chiếu với chúng giúp định vị rõ hơn hướng đi mà DocVault chọn. Bảng 2.1 so sánh DocVault với ba đại diện tiêu biểu trên các tiêu chí liên quan trực tiếp đến phạm vi đề tài; phần so sánh này dựa trên tài liệu và đặc điểm công khai của các sản phẩm chứ không phải kết quả kiểm thử trực tiếp của nhóm, nên chỉ mang tính định vị tương đối.

- **Microsoft SharePoint:** nền tảng quản lý tài liệu và cộng tác thương mại, mạnh về tích hợp hệ sinh thái Microsoft 365, phân quyền và compliance; tuy nhiên là dịch vụ đóng, chủ yếu vận hành trên đám mây của nhà cung cấp và khó tùy biến sâu chính sách truy cập ở mức mã nguồn.
- **Alfresco:** hệ thống ECM mã nguồn mở (có bản thương mại), hỗ trợ vòng đời tài liệu, phân quyền và audit; kiến trúc nghiêng về monolith/Java truyền thống và không đặt trọng tâm vào quy trình DevSecOps đi kèm.
- **Nextcloud:** giải pháp self-hosted phổ biến cho lưu trữ và chia sẻ file, dễ triển khai; mạnh về chia sẻ và đồng bộ nhưng kiểm soát theo classification, DLP và audit tamper-evident không phải trọng tâm mặc định.

Bảng 2.1: So sánh DocVault với một số hệ thống quản lý tài liệu tiêu biểu

Tiêu chí	SharePoint	Alfresco	Nextcloud	DocVault
Mô hình triển khai	Cloud đóng	Self-hosted	Self-hosted	Self-hosted
RBAC + ACL deny-overrides	Có	Có	Một phần	Có
Phân loại + kiểm soát theo classification	Có	Một phần	Hạn chế	Có
DLP nâng classification tự động	Một phần	Hạn chế	Hạn chế	Có
Audit tamper-evident (hash-chain)	Hạn chế	Hạn chế	Hạn chế	Có
Compliance boundary (metadata-only)	Hạn chế	Hạn chế	Hạn chế	Có
DevSecOps pipeline đi kèm	Không	Không	Không	Có

Điểm khác biệt của DocVault không nằm ở việc thay thế các hệ thống trên về quy mô tính năng, mà ở cách gộp ba yếu tố trong cùng một đề tài: (1) kiểm soát truy cập nhiều lớp theo classification với ranh giới compliance metadata-only, (2) audit tamper-evident bằng hash-chain, và (3) đưa toàn bộ quy trình phát hành vào pipeline DevSecOps có bằng chứng. Hướng tiếp cận này bám sát mục tiêu học thuật của đề tài: minh họa cách bảo mật được đưa xuyên suốt từ thiết kế ứng dụng đến quy trình triển khai, thay vì chỉ dựng một DMS đầy đủ tính năng.

2.3 Kiến trúc microservices

Microservices chia hệ thống thành nhiều service nhỏ, mỗi service sở hữu một năng lực nghiệp vụ và phát triển, kiểm thử, triển khai tương đối độc lập. DocVault dùng cách tách

này để phân định các bounded context:

- Gateway chịu trách nhiệm xác thực, routing và truyền ngữ cảnh.
- Metadata-service sở hữu metadata, ACL, trạng thái và policy.
- Document-service sở hữu upload/download file và tích hợp object storage.
- Workflow-service sở hữu state machine phê duyệt.
- Audit-service sở hữu audit trail và hash-chain.
- Notification-service nhận thông báo nghiệp vụ trong môi trường demo/dev.

Lý do tách quan trọng nhất với một hệ thống bảo mật là kéo lưu trữ file ra khỏi nơi quyết định quyền. File thật nằm ở MinIO/S3, còn quyền truy cập nằm ở metadata-service. Khi người dùng muốn tải file, document-service không tự ý quyết định mà phải dựa vào policy hoặc grant token do metadata-service cấp và xác minh lại, nên không service nào một mình nắm trọn cả file lẫn quyền.

2.4 Xác thực và phân quyền

2.4.1 Keycloak, OIDC và JWT

Keycloak là hệ thống quản lý định danh và truy cập mã nguồn mở, hỗ trợ các chuẩn phổ biến như OAuth 2.0 và OpenID Connect **keycloak_docs**. Trong DocVault, Keycloak được sử dụng để quản lý người dùng, vai trò, nhóm và phát hành token. Sau khi đăng nhập, người dùng nhận JWT chứa thông tin định danh và các claim cần thiết như user id, username, roles và groups.

Gateway và các service phía sau dùng JWT để xác định actor của request. Gateway là entry point chính, xác thực token và truyền các header ngữ cảnh như **x-user-id**, **x-roles**, **x-groups** xuống các service nội bộ. Nhờ đó service nghiệp vụ không phụ thuộc trực tiếp vào phiên đăng nhập frontend mà vẫn đủ thông tin để thực thi chính sách.

2.4.2 RBAC

RBAC gán quyền cho vai trò thay vì gán trực tiếp cho từng người dùng, rồi người dùng nhận một hoặc nhiều vai trò. Trong DocVault có năm vai trò:

- **viewer**: đọc tài liệu đã xuất bản nếu policy cho phép.
- **editor**: tạo tài liệu, chỉnh sửa metadata, upload file và gửi duyệt.
- **approver**: xem và phê duyệt hoặc từ chối tài liệu đang chờ duyệt.
- **compliance_officer**: xem audit, metadata và evidence trong phạm vi compliance, nhưng không được truy cập nội dung file.
- **admin**: quản trị hệ thống và có quyền rộng nhất.

RBAC đủ để kiểm soát quyền ở mức route hay chức năng, nhưng nó dừng lại ở mức vai trò: nó không nói được rằng riêng tài liệu này thì người dùng A được tải còn vai trò editor nói chung thì không. Đó là lý do DocVault bổ sung ACL ở mức từng tài liệu.

2.4.3 ACL và deny-overrides

ACL gán rule truy cập cho từng tài liệu. Một rule áp cho user, role, group hoặc tất cả, mang permission như `READ`, `DOWNLOAD`, `WRITE`, `APPROVE` và effect `ALLOW` hoặc `DENY`.

Khi nhiều rule cùng khớp, DocVault chọn chiến lược deny-overrides: chỉ cần có một rule `DENY` phù hợp thì kết quả là từ chối, kể cả khi tồn tại rule `ALLOW` khác. Nhóm chọn hướng này thay vì allow-overrides vì với tài liệu `SECRET`, một lệnh cấm bị ghi đè nguy hiểm hơn nhiều so với một quyền cho phép bị bỏ sót; khi không chắc, hệ thống nên nghiêng về từ chối.

2.5 Phân loại tài liệu và kiểm soát nội dung

DocVault sử dụng bốn mức phân loại tài liệu:

- **PUBLIC**: tài liệu công khai trong phạm vi người dùng đã xác thực.
- **INTERNAL**: tài liệu nội bộ.
- **CONFIDENTIAL**: tài liệu mật, cần kiểm soát preview/download nghiêm ngặt hơn.
- **SECRET**: tài liệu rất nhạy cảm, chỉ một số vai trò/đối tượng được phép truy cập.

Mức phân loại không dừng ở metadata hiển thị mà còn ảnh hưởng đến chính sách truy cập. Chẳng hạn, tài liệu `CONFIDENTIAL` hoặc `SECRET` có thể không trả về presigned URL trực tiếp mà phải đi qua endpoint stream, để document-service áp thêm kiểm soát như watermark posture và tái xác thực grant.

2.6 Audit, hash-chain và compliance evidence

Audit log ghi lại các sự kiện quan trọng trong hệ thống: tạo tài liệu, upload file, submit, approve, reject, download authorized/denied, DLP detected, malware blocked, retention archive và các thao tác compliance. Một audit log hữu ích thì ngoài việc có dữ liệu còn phải cho phép kiểm tra tính toàn vẹn.

DocVault dùng hash-chain cho audit events. Mỗi event mang một hash phụ thuộc vào nội dung của chính nó và hash của event trước. Nếu ai đó sửa thẳng một event cũ trong storage, các hash phía sau sẽ lệch và verify-chain phát hiện được. Cần nói rõ giới hạn: cơ chế này không biến audit log thành lưu trữ bất biến tuyệt đối, nó chỉ làm cho việc sửa đổi để lại dấu vết, tức tamper-evident, và mức đó là đủ cho phạm vi đồ án.

Ngoài audit query, hệ thống còn xuất evidence packet gom metadata, version checksum, workflow history, retention evidence, trạng thái verify-chain và các audit event liên quan đến một tài liệu. Một chi tiết được giữ rất kỹ: evidence packet không bao giờ chứa nội dung file, presigned URL, grant token hay object key nhạy cảm.

2.7 DevSecOps

DevSecOps đưa bảo mật vào trong quá trình phát triển và vận hành thay vì để dành kiểm tra ở cuối dự án. Ý tưởng là rải các kiểm soát ra nhiều điểm của pipeline: mã nguồn, thư viện phụ thuộc, hạ tầng, Docker image, ứng dụng sau triển khai, và giữ lại artifact làm bằng chứng. Cái lợi không chỉ là phát hiện sớm mà còn là tính lặp lại: mỗi lần thay đổi đều đi qua cùng một bộ kiểm tra.

Ở DocVault, tinh thần đó được hiện thực bằng Jenkins pipeline. Ngoài chạy test, pipeline còn chạy SAST, SCA, Trivy filesystem scan, Trivy image scan, Checkov IaC scan, Terraform validate, smoke test và DAST bằng OWASP ZAP. Các bước này bám theo tư tưởng “shift-left security” trong các tài liệu DevSecOps **owasp_devsecops**, **nist_ssdf**.

2.8 Các nhóm kiểm thử bảo mật trong pipeline

Bảng 2.2: Các nhóm kiểm thử bảo mật trong DevSecOps pipeline

Nhóm	Mục tiêu	Ví dụ áp dụng trong DocVault	Công cụ
SAST	Phân tích mã nguồn tĩnh để phát hiện lỗi chất lượng và bảo mật.	Kiểm tra backend/frontend TypeScript, quality gate và code smell.	SonarQube
SCA	Phân tích thư viện phụ thuộc và lỗ hổng đã biết.	Quét dependency của monorepo Node.js.	OWASP Dependency Check
FS scan	Quét filesystem repository.	Phát hiện lỗ hổng trong file hệ thống, lockfile, cấu hình.	Trivy
Image scan	Quét Docker image sau build.	Phát hiện CVE trong image backend/frontend.	Trivy
IaC scan	Quét cấu hình hạ tầng.	Kiểm tra Terraform/Kubernetes manifest.	Checkov
DAST	Kiểm thử ứng dụng đang chạy.	Baseline scan web app sau deploy.	OWASP ZAP

2.9 GitOps và Argo CD

GitOps lấy Git làm nguồn sự thật cho trạng thái triển khai. Khi pipeline build xong image mới, thay vì chạy lệnh sửa cluster thủ công, nó cập nhật Helm values hoặc manifest trong một branch GitOps; Argo CD theo dõi branch đó và đồng bộ trạng thái mong muốn xuống Kubernetes **argocd__docs**. Khác biệt thực tế là không ai `kubectl apply` trực tiếp lên cụm, mọi thay đổi triển khai đều đi qua một commit.

DocVault tận dụng đúng điểm này. Pipeline cập nhật các file values theo từng service, rồi Argo CD sync các application gateway, metadata-service, document-service, workflow-service, audit-service, notification-service và web. Việc tách build pipeline khỏi deployment controller giúp mỗi bên giữ một trách nhiệm, đồng thời để lại lịch sử thay đổi triển khai ngay trong Git.

2.10 Tổng hợp công nghệ và công cụ

Để có cái nhìn tổng thể, phần này tổng hợp các công nghệ và công cụ chính được sử dụng trong đề tài, chia thành hai nhóm: nhóm xây dựng ứng dụng (Bảng 2.3) và nhóm quy trình DevSecOps cùng hạ tầng triển khai (Bảng 2.4). Các phiên bản được ghi nhận theo cấu hình thực tế trong repository tại thời điểm hoàn thành báo cáo.

Bảng 2.3: Công nghệ xây dựng ứng dụng DocVault

Thành phần	Công nghệ	Phiên bản	Vai trò
Frontend	Next.js, React	16.2, 19.2	Giao diện web theo App Router.
UI & form	Tailwind CSS, Radix UI, React Hook Form, Zod	4.x, 7.7, 4.3	Thiết kế giao diện, form và validation phía client.
Server state	TanStack Query	5.90	Quản lý dữ liệu server, cache và đồng bộ.
Backend	NestJS	10.0	Framework cho gateway và các microservice.
Ngôn ngữ	TypeScript	5.x	Ngôn ngữ chính cho frontend và backend.
Xác thực	Keycloak, passport-jwt, jwks-rsa	—	OIDC/JWT, xác minh token bằng JWKS RS256.
ORM	Prisma + adapter-pg	7.5	Truy cập PostgreSQL bằng driver adapter.
Metadata DB	PostgreSQL	—	Lưu metadata, ACL, workflow history.
Audit DB	MongoDB, Mongoose	—	Lưu audit event và hash-chain.
Object storage	MinIO / Amazon S3	—	Lưu blob file tài liệu.
Rate limit	@nestjs/throttler	6.5	Giới hạn tần suất request theo cấu hình chung.
Monorepo	pnpm, Turbo	9.15, 2.1	Quản lý workspace và điều phối build.

Bảng 2.4: Công cụ DevSecOps và hạ tầng triển khai

Nhóm	Công cụ	Vai trò trong đề tài
CI/CD	Jenkins, Jenkins Shared Library	Điều phối pipeline, tách CI validation và CD release.
Secret scan	TruffleHog	Phát hiện credential bị commit trong repository.
SAST	SonarQube	Phân tích mã nguồn tĩnh và Quality Gate.
SCA	OWASP Dependency-Check	Quét lỗ hổng thư viện phụ thuộc theo CVSS.
FS/Image scan	Trivy	Quét filesystem và Docker image.
IaC scan	Checkov	Quét cấu hình Terraform, Kubernetes, Helm.
Policy-as-code	Kyverno CLI	Kiểm tra policy trên manifest đã render.
IaC	Terraform	Mô tả hạ tầng AWS dưới dạng mã.
Container	Docker BuildKit	Build và cache image theo batch.
Registry	Harbor	Private registry, lưu SBOM và trạng thái ký.
Ký artifact	Cosign	Ký và xác minh image digest.
GitOps	Git, Argo CD	Đồng bộ desired state theo mô hình app-of-apps.
Orchestration	Amazon EKS, Helm, Kustomize	Chạy workload trên Kubernetes.
Secret & key	AWS Secrets Manager, External Secrets, KMS	Quản lý bí mật và khóa mã hóa (customer-managed key, xoay vòng hằng năm).
DAST	OWASP ZAP	Quét động ứng dụng sau triển khai.
Observability	Prometheus, Grafana, Loki	Thu thập metric, log và trực quan hóa.

Chương 3

Phân tích yêu cầu và thiết kế tổng thể

3.1 Tổng quan bài toán

DocVault hướng đến bài toán quản lý tài liệu trong môi trường có nhiều vai trò và nhiều mức độ nhạy cảm dữ liệu. Một tài liệu đi qua nhiều trạng thái: bản nháp, chờ duyệt, đã xuất bản và đã lưu trữ. Trong quá trình đó, mỗi vai trò chỉ được thực hiện những thao tác phù hợp. Editor có thể tạo và gửi duyệt tài liệu; Approver có thể phê duyệt hoặc từ chối; Viewer chỉ được truy cập tài liệu đã xuất bản nếu policy cho phép; Compliance Officer có thể xem audit và evidence nhưng không được truy cập nội dung file; Admin có quyền quản trị rộng hơn.

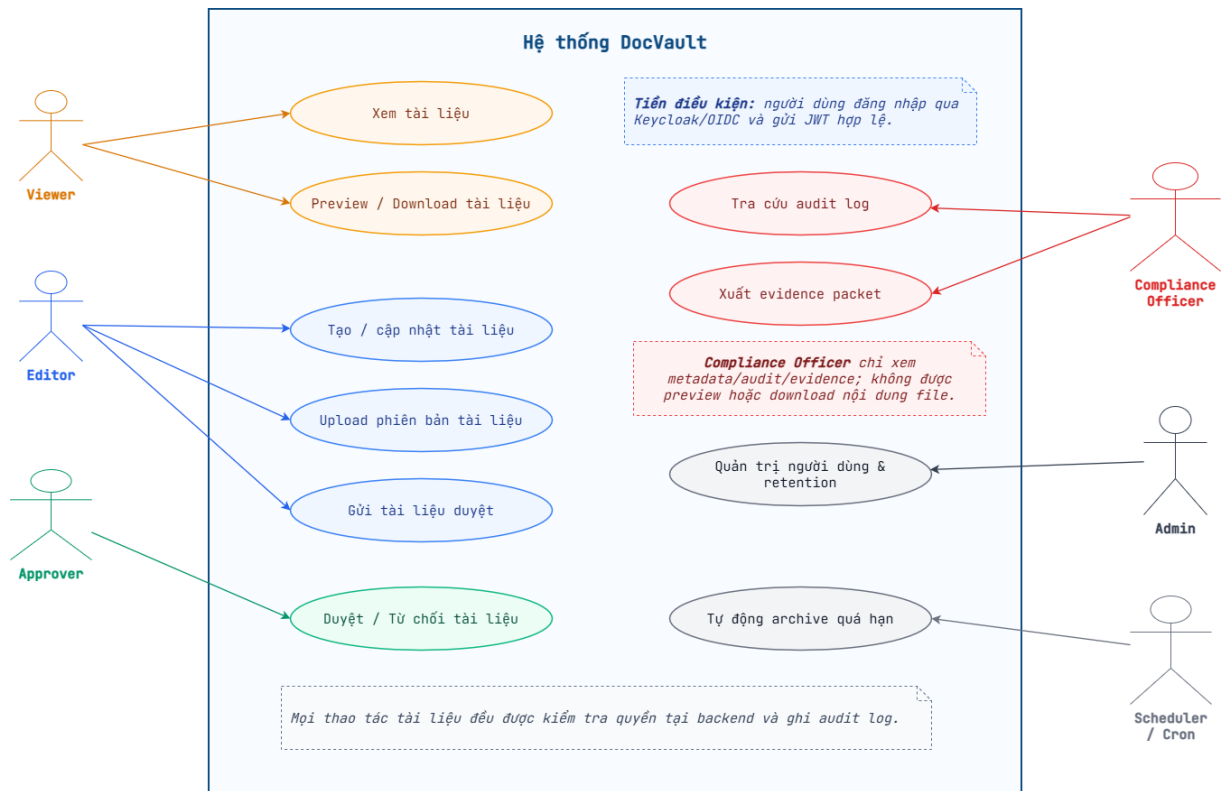
Bài toán ở đây không dừng ở việc lưu file, mà là kiểm soát file theo ngữ cảnh. Cùng một file có thể được phép preview với người này nhưng bị từ chối với người khác; có thể được cấp presigned URL khi là tài liệu public nhưng phải stream qua backend khi là tài liệu confidential; có thể cho xem metadata nhưng không cho tải nội dung. Vì vậy, hệ thống cần một mô hình policy đủ rõ và được thực thi ở backend.

3.2 Tác nhân hệ thống

Bảng 3.1: Các tác nhân chính trong DocVault

Tác nhân	Vai trò trong hệ thống
Viewer	Người dùng đọc tài liệu đã được xuất bản trong phạm vi policy cho phép; có thể preview/download tài liệu public hoặc tài liệu được cấp quyền.
Editor	Người tạo và quản lý tài liệu của mình; có thể tạo draft, chỉnh metadata, upload phiên bản, gửi duyệt và archive khi có quyền.
Approver	Người kiểm tra tài liệu đang ở trạng thái chờ duyệt; có thể approve hoặc reject theo quy trình.
Compliance Officer	Người kiểm tra tuân thủ; được xem audit, metadata/evidence được phép, nhưng bị chặn preview, stream, presign và download nội dung file.
Admin	Người quản trị hệ thống; có quyền rộng hơn trên tài liệu, thành viên, evidence và retention.
Scheduler / Cron	Tác nhân hệ thống (không phải người dùng); chạy nền theo lịch để tự động archive tài liệu đến hạn retention và dọn tài liệu trong thùng rác quá hạn lưu giữ.

Hình 3.1 tổng hợp các use case chính của từng tác nhân trong một sơ đồ thống nhất. Ngoài bốn vai trò người dùng, sơ đồ bổ sung tác nhân Scheduler/Cron đại diện cho các tác vụ nền theo lịch. Sơ đồ làm rõ ba điểm thiết kế cốt lõi của DocVault: mọi tác nhân phải đăng nhập qua Keycloak/OIDC trước khi thao tác; mọi thao tác trên tài liệu đều được kiểm tra quyền tại backend và ghi audit log; và Compliance Officer dù xem được metadata, audit, evidence vẫn luôn bị từ chối preview hoặc download nội dung file.



Hình 3.1: Sơ đồ use case tổng thể của DocVault theo tác nhân

3.3 Yêu cầu chức năng

Các yêu cầu chức năng được nhóm theo miền nghiệp vụ như sau:

3.3.1 Xác thực và phân quyền

Hệ thống cần hỗ trợ đăng nhập thông qua Keycloak, đọc thông tin người dùng hiện tại và phân quyền theo vai trò. Navigation, command palette và các thao tác trên UI cần thay đổi theo role, nhưng backend vẫn phải là nơi quyết định cuối cùng. Các request không có token, token hết hạn hoặc token không hợp lệ phải bị từ chối.

3.3.2 Quản lý tài liệu

Người dùng có quyền có thể tạo tài liệu mới với tiêu đề, mô tả, classification và tags. Sau khi tạo, tài liệu ở trạng thái DRAFT. Editor hoặc Admin có thể upload file ban đầu hoặc upload phiên bản mới. Hệ thống cần lưu checksum, kích thước, MIME type, tên file và người upload cho từng version.

3.3.3 Workflow phê duyệt

Tài liệu đi qua vòng đời chính DRAFT -> PENDING -> PUBLISHED -> ARCHIVED. Editor gửi tài liệu từ Draft sang Pending. Approver phê duyệt tài liệu Pending sang Published hoặc từ chối để quay về Draft. Tài liệu Published có thể được archive theo quyền. Mỗi transition cần ghi workflow history và audit event.

3.3.4 Preview và download

Người dùng có quyền có thể preview hoặc download tài liệu tùy trạng thái, classification, role, ownership và ACL. Với tài liệu nhạy cảm, hệ thống có thể không trả về URL tải trực tiếp mà trả về endpoint stream có kiểm soát. Compliance Officer luôn bị chặn truy cập nội dung file.

3.3.5 Audit, security và evidence

Hệ thống cần ghi audit event cho các hành động quan trọng, hỗ trợ truy vấn audit theo bộ lọc, kiểm tra hash-chain và hiển thị security summary. Compliance/Admin có thể xuất evidence packet liên quan đến tài liệu, recommendation hoặc compliance flow.

3.3.6 Retention và access review

Tài liệu đã xuất bản cần có retention evidence dựa trên classification. Admin có thể chạy retention job demo để auto-archive các tài liệu đến hạn. Compliance/Admin có thể xem access review cho tài liệu nhạy cảm, broad ACL grant hoặc quyền đã cũ.

3.4 Yêu cầu phi chức năng

Bảng 3.2: Yêu cầu phi chức năng

Nhóm yêu cầu	Mô tả
Bảo mật	Backend phải thực thi policy; token và grant phải được xác minh; nội dung file không được lộ qua evidence hoặc audit.
Truy vết	Các thao tác quan trọng cần tạo audit event; audit chain cần có khả năng verify.
Tách biệt trách nhiệm	Mỗi service sở hữu một bounded context rõ ràng; document-service không tự quyết định policy download.
Khả năng kiểm thử	Có unit test, E2E runtime, frontend test/lint/build và pipeline scan.
Khả năng triển khai	Hỗ trợ local Docker Compose, Docker image, Kubernetes/Helm và GitOps.
Khả năng mở rộng	Kiến trúc cho phép bổ sung message queue, kênh thông báo, môi trường triển khai và replica độc lập; nền tảng hiện tại đã tích hợp SBOM, ký image, KMS, External Secrets và policy-as-code trong CI.

3.5 Mô hình threat cơ bản

Để định hướng thiết kế bảo mật, hệ thống xem xét các nhóm rủi ro sau. Mỗi nhóm được phân loại theo khung STRIDE **microsoft_stride** để làm rõ bản chất mối đe dọa và liên kết với biện pháp giảm thiểu tương ứng trong DocVault.

Bảng 3.3: Mô hình threat của DocVault ánh xạ theo STRIDE

Rủi ro	Loại STRIDE	Biện pháp giảm thiểu trong DocVault
Truy cập trái phép metadata	Information Disclosure	Người dùng đoán document id và gọi API chi tiết, workflow history, comments hoặc ACL; backend policy-filter theo role, owner, ACL, classification và status.
Tải nội dung file trái phép	Elevation of Privilege	Gọi trực tiếp presign, stream hoặc download endpoint mà không có quyền; chặn bằng grant token ngắn hạn và tái xác minh ở document-service.
Lạm dụng Compliance Officer	Elevation of Privilege	Vai trò compliance xem được nhiều metadata nhưng luôn bị deny nội dung file ở mọi classification.
Upload file độc hại	Tampering	File EICAR/malware bị chặn trước khi ghi MinIO và không tạo version metadata.
Rò rỉ dữ liệu nhạy cảm	Information Disclosure	DLP phát hiện email, số điện thoại, từ khóa nhạy cảm trong tài liệu public/internal và nâng classification.
Downgrade classification không hợp lệ	Tampering	Tài liệu đã bị DLP phát hiện không được hạ xuống public/internal nếu thiếu quyền và lý do override hợp lệ.
Giả mạo audit event	Spoofing	Người dùng thường không append audit trực tiếp; audit ingest yêu cầu service token.
Sửa audit log cũ	Tampering	Hash-chain phát hiện khi audit event bị thay đổi trực tiếp trong storage.

Phần lớn rủi ro tập trung vào hai loại Information Disclosure và Elevation of Privilege, phản ánh đúng đặc thù của một hệ thống quản lý tài liệu nhạy cảm: mối đe dọa chính là lộ dữ liệu và vượt quyền truy cập. Hai loại Tampering và Spoofing liên quan đến tính toàn vẹn của file và audit trail. Các nhóm Repudiation và Denial of Service không phải trọng tâm trong phạm vi đồ án; tính chống chối bỏ được hỗ trợ một phần qua audit hash-chain, còn chống từ chối dịch vụ được xử lý ở mức cơ bản bằng rate limiting.

3.6 Kiến trúc tổng thể

Kiến trúc DocVault gồm bốn lớp: frontend, gateway, microservices và hạ tầng dữ liệu. Frontend Next.js là giao diện người dùng. API Gateway là điểm vào duy nhất cho frontend, chịu trách nhiệm xác thực, role guard, proxy request và truyền ngữ cảnh. Các service phía sau xử lý nghiệp vụ riêng. Hạ tầng dữ liệu gồm Keycloak, PostgreSQL, MongoDB

và MinIO/S3.

Hình ?? mô tả cách các lớp này liên kết với nhau. Trong kiến trúc đó, frontend chỉ giao tiếp qua gateway, còn các microservice phía sau giữ ranh giới nghiệp vụ riêng: metadata-service sở hữu policy và metadata, document-service sở hữu blob file, workflow-service điều phối trạng thái, audit-service ghi nhận sự kiện và notification-service xử lý thông báo.